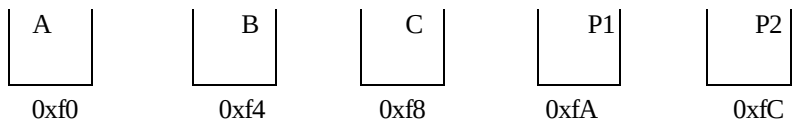
Larbi Tebessi University- Tebessa
Department of mathematics and computer science, L1

**Algorithms and data structure 2**

# TD/TP 2

| | |
|---|---|
| Reminder: A list type of integers can be declared as follows:<br>Type Element = Structure<br>Val: Integer<br>Next: *Element<br>End-Structure<br>List = *Element<br>Var L, P: List /* Two variables of type List */ | In a C program, the List type (of integers) can be declared as follows:<br>typedef struct Element Element;<br>typedef Element* List;<br><br>struct Element {<br>int val; // val contains useful information<br>List next; // next points to the next element<br>};<br>List L, P; // L and P are pointers to Element |
| Pointer Operations<br>Initialization: P ← Nil. Nil: is a constant representing a particular address that does not point to any data.<br>Allocate(P): allows creating a new element (or cell) pointed to by pointer P.<br>Free(P): allows deleting the element (or cell) pointed to by pointer P. | In C:<br>Initialization: P=NULL;<br>Creation: P = malloc(sizeof(Element));<br>Freeing (destruction): free(p); |

**Exercice 1 (TD/TP)** :_Let three integers A, B, and C, and two integer pointers P1 and P2.




1. Determine the value of the different elements given in the table for each operation.

| | A | B | C | &A | &B | &C | P1 | P2 | *P1 | *P2 |
|---|---|---|---|---|---|---|---|---|---|---|
| **A=1, B=2, C=3<br>P1=&A, P2=&B** | | | | | | | | | | |
| **P2= &C** | | | | | | | | | | |
| ***P1 = *P2** | | | | | | | | | | |
| **(*P2)++** | | | | | | | | | | |
| **P1= P2** | | | | | | | | | | |
| **P2= &B** | | | | | | | | | | |
| ***P2 = *P1 - 2 * *P2** | | | | | | | | | | |
| **(*P2)--** | | | | | | | | | | |
| **C = (P2 == &C)** | | | | | | | | | | |
| ***P2= *P1 + A** | | | | | | | | | | |

2. Confirm the results of this table using a C program by implementing a display procedure with global variables (the actual addresses will change, of course).

**Exercise 2 (TD):** 1. Declare pointer variables for: an integer, a string, an array of integers (10 max). 2. Create corresponding dynamic variables. Assign values to each dynamic variable pointed to by these pointers.

**Exercice 3 (TD):** We consider that the following declarations have been made:

Var A : integer
tab : array [10] of char

An expression with pointers (or without pointers) is given to you; you must rewrite it without (or with) the explicit use of pointers.

1. (@ a)*
2. tab*
3. (tab + 0)*
4. (tab*) + 1
5. @(tab[0])
6. @(tab[i])
7. tab[i]←tab[i]+1

**Exercise4: (TD/TP)** Write a program that:

- Dynamically allocates an array `T` of real numbers (its size is entered by the user).
- Fills the dynamic array `T` with real numbers.
- Creates two other dynamic arrays:
  - `TP` to store all positive numbers,
  - `TN` to store all negative numbers.
- Leaves the zero values unchanged (they remain only in `T` and are not stored in `TP` or `TN`).

**Exercise 5 (TD/TP):** Write an algorithm then C program that:

1. Defines a `structure Element` containing:
   - an `integer val`
   - a pointer to the next element.
2. Creates a first element with value `10`.
3. Adds a second element at the beginning with value `20`.
4. Asks the user to enter an integer, and adds this third element at the end of the list.
5. Displays the list content.
6. Frees all dynamically allocated memory.

**Exercise 6 (TD/TP):** We use linked lists throughout this exercise, the type of which is as follows: Type Cell = Structure Info: Integer Next: *Cell End-Structure List = *Cell;

Q1) Write a procedure InsertHead(L, N) which allows inserting integer N at the beginning of list L (List L can be empty).

Q2) Use the InsertHead procedure to write another procedure CreateList which allows creating a list of positive integers from the keyboard. If a negative value is entered, insertion into the list stops.

Q3) Write a procedure DisplayList(L) which allows displaying the elements of list L. If L is empty, the procedure should display the message '* Empty list *'.

Q4) Write the main algorithm that allows testing the procedures written in the previous questions. Q5) Write a procedure InsertTail(L, N) which allows inserting integer N at the end of list L (List L can be empty).

Q6) Modify the procedure written in question Q2 to use the InsertTail procedure.

Q7) Write a function Length(L) which allows calculating the length (number of elements) of list L.

Q8) Write a recursive procedure DisplayListREC(L) which allows displaying the elements of list L.

Q9) Modify the procedure DisplayListREC(L) to display the elements of list L in reverse order.

Q10) Rewrite the Length function and the InsertTail procedure in recursive form.

Q11) It is assumed, this time, that the elements of the list are sorted. Write a procedure InsertSorted(L, N) which allows inserting an integer N so that the list always remains sorted. Note: This sorted list accepts duplicates.

**Exercise 7 (TD/TP):** Write algorithms (in pseudocode) then the c program for the following stack operations:

1. Push(S, x): Add element `x` to the top of the stack `S`.
2. Pop(S): Remove and return the top element of the stack.
3. IsEmpty(S): Return `true` if the stack is empty.
4. IsFull(S): Return `true` if the stack is full.
5. Display(S): Print all elements of the stack from top to bottom.

**Exercise 8 (TD/TP):** Write pseudocode then the program C for the following operations:

1. Enqueue(Q, x): Add element `x` to the rear of the queue `Q`.
2. Dequeue(Q): Remove and return the front element of the queue.
3. IsEmpty(Q): Check if the queue is empty.
4. IsFull(Q): Check if the queue is full.
5. Display(Q): Print the queue from front to rear.

**Exercise 9 (TP):** Suppose you need to implement a doubly linked list in C. You must perform the following operations:

6. insertFirst(): Inserts a new element at the beginning of the list. insertLast(): Inserts a new element at the end of the list. deleteFirst(): Deletes the first element of the list. deleteLast(): Deletes the last element of the list. isEmpty(): Checks if the list is empty. display(): Displays the elements of the list from left to right and from right to left.
7. Make sure to define an appropriate structure to represent the nodes of the doubly linked list. Each node should contain fields for the data as well as pointers to the previous and next nodes.
8. Ensure that you handle cases where the list is empty or when you insert or delete elements at the beginning or end of the list. Try implementing these operations and testing them with different scenarios to ensure they work correctly.

**Exercise home work:** Write a function tabToList which transforms an array of integers into a linked list. The function must return a pointer that indicates the head of the list. - Write a function listToTab which transforms a linked list of integers into an array. The function must return an array that contains all the elements of the linked list.

**Exercise home work:** Write a procedure that allows sorting a linked list Ls in ascending order of elements.

**Exercise home work:** Write a function in the C language that takes a string containing parentheses, brackets, and braces and checks if the order of parentheses is correctly balanced. For example: The input "{[()]}" is balanced. The input "{[(])}" is not balanced. Use a stack to solve this problem.

**Exercise home work:** Suppose you have a waiting queue represented by a FIFO (First-In-First-Out) data structure. You must implement the following operations: enqueue(): Adds an element to the end of the queue. dequeue(): Removes the element at the head of the queue and returns it. isEmpty(): Checks if the queue is empty. front(): Returns the element at the head of the queue without removing it. size(): Returns the number of elements in the queue. Use an array-based implementation to store the elements of the queue.